00 주소

'배열명'고 '&배열명'이 의미하는 바에는 차이가 있다. '배열명'은 배열의 첫 번째 주소를, '&배열명'은 배열 전체의 주소를 의미한다.

예시 코드	출력

`#include <stdio.h>`
`int main(){`
`  int data[7]={1,2,3,4,5,6,7};`
 	
`  printf("1. %d\n", data);`
`  printf("2. %d\n", data+1);`
`  printf("3. %d\n", &data);`
`  printf("4. %d\n", &data+1);`
`}`
	1. 6487584
	2. 6487588
	3. 6487584
	4. 6487612

※ 포인터 연산을 하면 컴파일러가 자료형의 크기를 자동으로 반영한다. (주소1 – 주소2) ÷ (자료형 크기)

01 정렬

sort 함수의 정렬 범위는 [시작 위치, 끝 위치) 이다. 반환형은 void로, 아무것도 반환하지 않는다.

● 오름차순

구분	코드
헤더 파일	`#include <algorithm>`
비교 기준	생략 시, 디폴트값 – 오름차순
호출 방식	`std::sort( arr, arr + n )`

● 내림차순

구분	코드
헤더 파일	`#include <algorithm>`
비교 기준	`bool cmp( int a, int b ) { return a > b; }`
호출 방식	`std::sort( arr, arr + n, cmp )`

02 이분탐색

upper_bound 는 key 이상인 값이 처음 나오는 위치를, lower_bound 는 key 초과인 값이 처음 나오는 위치를 반환한다. upper_bound - lower_bound 는 key의 개수와 같다.

● **lower_bound** – key 이상인 값이 처음 등장하는 위치

구분	코드
헤더 파일	`#include <algorithm>`
호출 방식	`lower_bound( arr, arr + cnt, key ) // 해당 위치를 가리키는 포인터 반환`
활용 예시	`lower_bound( arr, arr + cnt, key ) - arr // 인덱스로 계산`

● **upper_bound** – key 초과인 값이 처음 등장하는 위치

구분	코드
헤더 파일	`#include <algorithm>`
호출 방식	`upper_bound( arr, arr + cnt, key ) // 해당 위치를 가리키는 포인터 반환`
활용 예시	`upper_bound( arr, arr + cnt, key ) - arr // 인덱스로 계산`

03 스택과 큐

스택과 큐는 자료를 꺼내는 순서만 서로 반대인 자료구조이다. pop()은 삭제만 수행하고 값을 반환하지 않으므로, 값이 필요하면 top() 또는 front()로 먼저 읽어야 한다

● **stack** – 마지막에 넣은 값이 먼저 나옴(LIFO)

구분	코드
헤더 파일	`#include <stack>`
호출 방식	`stack<int> S; S.push(v); S.top(); S.pop();  // 선언·삽입·확인·삭제`
활용 예시	`while(!S.empty()){ x=S.top(); S.pop(); }  // 끝까지 꺼냄`

● **queue** – 먼저 넣은 값이 먼저 나옴(FIFO)

구분	코드
헤더 파일	`#include <queue>`
호출 방식	`queue<int> Q; Q.push(v); Q.front(); Q.pop();  // 선언·삽입·확인·삭제`
활용 예시	`while(!Q.empty()){ x=Q.front(); Q.pop(); }  // 끝까지 꺼냄`





● ~

구분	코드
헤더 파일	
호출 방식	
활용 예시	

0* ~

~

● ~

구분	코드
헤더 파일	
호출 방식	
활용 예시	

● ~

구분	코드
헤더 파일	
호출 방식	
활용 예시	

#2 [도전]미로를 그래프로 표현하기

통로 칸에만 입력 순서대로 정점 번호를 부여한다. 정점 값(행*10+열)의 오름차순은 입력 순서와 같으므로, 이 번호를 그대로 인접행렬의 인덱스로 사용할 수 있다. 각 통로 칸에서 상하좌우 네 방향을 검사하여 인접한 칸이 통로이면 graph[a][b]=1;로 연결한다. 이웃 칸을 검사하기 전에 미로 범위를 벗어나지 않는지 먼저 확인하고, 출력할 때는 정점 0번부터 cnt-1번까지의 행과 열만 출력한다.

```c
#include <stdio.h>
int n, maze[5][5], id[5][5], graph[25][25], cnt;
int dx[4]={-1, 1, 0, 0};
int dy[4]={0, 0,-1, 1};

int main(){
    scanf("%d", &n);
    // 미로 입력 및 정점 번호 부여: 통로(0) 칸에 입력 순서대로 0, 1, 2, … 번호 부여
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++){
            scanf("%d", &maze[i][j]);
            if(maze[i][j]==0) id[i][j] = cnt++;
        }
    // 인접행렬 구성: 각 통로 칸에서 상하좌우 네 방향을 검사하여, 인접한 칸이 통로면 정점 연결
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++){
            if(maze[i][j]==1) continue;                    // 벽은 정점에서 제외
            for(int d=0; d<4; d++){
                int x=i+dx[d], y=j+dy[d];                  // 인접 칸의 좌표
                if(x<0 || x>=n || y<0 || y>=n) continue;   // 미로 범위 이탈
                if(maze[x][y]==1) continue;                // 벽
                graph[id[i][j]][id[x][y]]=1;
            }
        }
    // 정점 0번부터 cnt-1번까지 행렬 출력
    for(int i=0; i<cnt; i++){
        for(int j=0; j<cnt; j++)
            printf("%d ", graph[i][j]);
        printf("\n");
    }
}
```



#2 stack으로 2진수 변환하기

2로 나눈 나머지를 차례로 스택에 push한 뒤 pop하면, 나머지를 거꾸로 읽는 이진수 변환 순서가 자연스럽게 만들어진다. 입력이 0 이면 반복문이 한 번도 실행되지 않아 아무것도 출력되지 않을 수 있으므로, while 대신 do-while을 사용해 최소 한 번은 push하도록 한다.

```
#include <stdio.h>
#include <stack>
using namespace std;

stack<int> S;

int main(){
    int n;
    scanf("%d", &n);

    // 나머지를 스택에 저장
    do{
        S.push(n%2);
        n /= 2;
    }while(n > 0);

    // 스택에서 꺼내어 출력
    while(!S.empty()){
        printf("%d", S.top());
        S.pop();
    }
}
```

#2 깊이우선탐색 2

간선을 입력받는 대로 vector에 push_back하면 인접 정점이 입력 순서대로 저장되므로, 목록을 앞에서부터 탐색하면 된다. 만약 인접행렬로 구현한다면, 간선의 입력 순번을 저장해 두고 매번 순번이 가장 작은 이웃을 찾는 과정이 필요하다.

```
#include <stdio.h>
#include <vector>
using namespace std;

vector<int> graph[11];   // graph[정점]=인접 정점 목록(입력 순서대로 저장)
bool visited[11];        // 방문 여부
int n, m;

void dfs(int k){
    printf("%d ", k);        // 현재 정점 방문
    visited[k] = true;       // 방문 표시
    // 인접 정점 탐색: 목록에 저장된 순서(간선 입력 순서)대로 검사
    for(int i=0; i<graph[k].size(); i++){
        int next = graph[k][i];
        if(!visited[next])
            dfs(next);
    }
}

int main(){
    scanf("%d %d", &n, &m);
    // 간선 입력: 양방향으로 인접리스트에 추가
    for(int i=0; i<m; i++){
        int a, b;
        scanf("%d %d", &a, &b);
        graph[a].push_back(b);
        graph[b].push_back(a);
    }
    dfs(1);   // 정점 1에서 출발
}
```

#2 너비우선탐색 2

간선을 vector에 push_back하면 입력 순서가 보존되므로, 꺼낸 정점의 목록을 앞에서부터 큐에 넣으면 된다. 만약 인접행렬로 구현한다면, 간선의 입력 순번을 저장해 두고 매번 순번이 가장 작은 이웃을 찾는 과정이 필요하다.

```
#include <stdio.h>
#include <vector>
#include <queue>
using namespace std;

vector<int> graph[11];   // graph[정점] = 인접 정점 목록 (입력 순서대로 저장)
queue<int> Q;
bool visited[11];        // 방문 여부
int n, m;

void bfs(int k){
    printf("%d ", k);        // 시작 정점 방문
    visited[k] = true;
    Q.push(k);
    // [2] 큐가 빌 때까지 반복: 꺼낸 정점의 인접 정점을 목록 순서대로 검사하여 큐에
넣는다.
    while(!Q.empty()){
        k = Q.front(); Q.pop();
        for(int i=0; i<graph[k].size(); i++){
            int next = graph[k][i];
            if(!visited[next]){
                printf("%d ", next);   // 새 정점 방문
                visited[next] = true;
                Q.push(next);
            }
        }
    }
}

int main(){
    scanf("%d %d", &n, &m);
    // [1] 간선 입력: 양방향으로 인접리스트에 추가한다.
    for(int i=0; i<m; i++){
        int a, b;
        scanf("%d %d", &a, &b);
        graph[a].push_back(b);
        graph[b].push_back(a);
    }
    bfs(1);   // 정점 1에서 출발
}
```